

Résolution de systèmes linéaires

Soit à résoudre le système $Ax=b$

Calcul de l'inverse de A ?

Une première idée est : Calculer A^{-1} puis $A^{-1}b$

Ce n'est pas une bonne idée car :

- Temps de calcul : calculer $A^{-1}c$ est résoudre n systèmes linéaires
- Espace mémoire : A très creuse et A^{-1} pleine

Formule de Cramer ?

Cramer : calcul $n+1$ déterminants Coût en $(n+1)!$

Bonnes méthodes Coût en $\frac{1}{3}n^3$

Pour $n=10$, on a 40 millions d'opérations pour Cramer contre 333 opérations !!

Méthodes directes les plus connues

- Méthode de Gauss (matrice quelconque)
- Méthode de Cholesky-Crout (matrice symétrique définie positive $\equiv$ valeurs propres strictement positives)
- Méthode de factorisation LU (matrice quelconque)

La matrice est d'abord factorisée en un produit de matrices triangulaires, avant de procéder à la résolution du système linéaire.

Lors de la factorisation, certains termes sont modifiés n fois (matrice dim $n \times n$) $\Rightarrow$ accumulation des erreurs de calculs.

L' algorithme effectue les mêmes opérations indépendamment des valeurs numériques manipulées $\Rightarrow$

un résultat est toujours obtenu.

La durée de calcul est en n^3 et ne dépend que de n .

Dans le cas de matrices mal conditionnées, la solution risque d' être erronée

Méthodes itératives les plus connues:

- Méthode du gradient (matrice symétrique définie positive)
- Méthode du gradient conjugué (matrice symétrique définie positive)
- Méthode du bi-gradient conjugué (matrice quelconque)

Partant qu'un vecteur $\mathbf{x}_0$, Il s'agit de construire une suite de vecteurs $\mathbf{x}_p$ de manière récurrente permettant de s'approcher à chaque itération de la solution

La procédure s'arrête d'après le critère :
$$\frac{\|[\mathbf{A}] \mathbf{x}_p - \mathbf{b}\|}{\|\mathbf{b}\|} \leq \varepsilon \approx 10^{-6}$$

L'algorithme peut ne pas converger s'il n'est pas adapté à la structure de la matrice ou si elle est mal conditionnée.

Mais si une solution est trouvée elle est nécessairement valide compte tenu de la précision relative demandée

Deux cas simples (et fondamentaux)

Matrice diagonale : $A=(a_{ij})$ avec $a_{ij} = \begin{cases} \neq 0 & \text{si } i = j \\ = 0 & \text{si } i \neq j \end{cases}$

Code Matlab de résolution :

x = b ./ D **%vec D = diag(A)**

Algébriquement : $x_k = b_k / a_{kk}$ pour $k = 1, 2, \dots, n$

Matrice triangulaire supérieure U : $A=(a_{ij})$ avec $a_{ij} = 0$ si $i > j$ (et $a_{ii} \neq 0$)

Méthode de remontée :

$$\begin{bmatrix} a_{1,1} & & & & a_{1,n} \\ 0 & \ddots & & & \\ & & a_{i,j} & & \\ 0 & 0 & a_{n-2,n-2} & a_{n-2,n-1} & a_{n-2,n} \\ 0 & 0 & 0 & a_{n-1,n-1} & a_{n-1,n} \\ 0 & 0 & 0 & 0 & a_{n,n} \end{bmatrix} \begin{bmatrix} x_1 \\ \\ \\ x_{n-2} \\ x_{n-1} \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ \\ \\ b_{n-2} \\ b_{n-1} \\ b_n \end{bmatrix}$$

Pour n : $x_n = b_n / a_{n,n}$

pour n-1 : $a_{n-1,n-1} x_{n-1} + a_{n-1,n} x_n = b_{n-1} \quad \Rightarrow \quad x_{n-1} = \frac{1}{a_{n-1,n-1}} (b_{n-1} - a_{n-1,n} x_n)$

Deux cas simples (et fondamentaux)

Matrice triangulaire supérieure **U** : Méthode de remontée

$$\begin{bmatrix} a_{1,1} & \cdots & \cdots & \cdots & a_{1,n} \\ 0 & \ddots & & & \vdots \\ 0 & 0 & \ddots & & \vdots \\ 0 & 0 & 0 & \ddots & \vdots \\ 0 & 0 & 0 & 0 & a_{n,n} \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_{n-2} \\ x_{n-1} \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ \vdots \\ b_{n-2} \\ b_{n-1} \\ b_n \end{bmatrix}$$

$$x_n = b_n / a_{n,n}$$

Pour k=n-1 à 1 (pas -1)

$$x_k = \frac{1}{a_{kk}} \left(b_k - \sum_{l=k+1}^{l=n} a_{kl} x_l \right)$$

Fin pour k

Code Matlab de résolution :

```
A = [1 2 3;  
     0 1 1;  
     0 0 2];  
  
b = [1 2 3]';
```

```
n = size(A,1);  
x = zeros(n,1);  
  
x(n) = b(n) / A(n,n);  
for k = n-1:-1:1  
    l=k+1:n;  
    v=A(k,l)*x(l);  
    x(k) = (b(k) - v) / A(k,k);  
end
```

Principe des méthodes directes

Etape 1 : **factorisation de la matrice A**

La matrice A est décomposée en un produit de matrices simples à inverser.

Par exemple $A=LU$ avec L et U triangulaires respectivement inférieure et supérieure.

Etape 2 : **résolution**

A partir de la factorisation (puisque c' est simple !).

$$Ax = b \Rightarrow LUx = b$$

En posant $y = Ux$

$$\Rightarrow \text{Résolution } Ly = b \Rightarrow y \quad (\text{Descente})$$

$$\text{Résolution } Ux = y \Rightarrow x \quad (\text{Montée})$$

Principe des méthodes directes

Matrice triangulaire inférieure L : méthode de descente

$$\begin{bmatrix} a_{1,1} & 0 & 0 & 0 & 0 \\ a_{2,1} & a_{2,2} & 0 & 0 & 0 \\ \vdots & & a_{ii} & 0 & 0 \\ \vdots & & & \ddots & 0 \\ a_{n,1} & \dots & \dots & \dots & a_{n,n} \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ \vdots \\ b_n \end{bmatrix}$$

$y_1 = b_1 / a_{1,1}$
 Pour $k=2$ à n

$$y_k = \frac{1}{a_{k,k}} \left(b_k - \sum_{l=1}^{k-1} a_{k,l} y_l \right)$$

 Fin pour k

Code Matlab de résolution :

```

n = size(A,1);
y = zeros(n,1);

A =[2 0 0;
    1 1 0;
    1 2 3];

b =[1 2 3]';

y(1)=b(1)/A(1,1); % A COMPLETER
    
```

Factorisation de Gauss LU

- Domaine de validité : **toutes matrices**
- Décomposition de $A=LU$ où
 - U matrice triangulaire supérieure,
 - L matrice triangulaire inférieure

$$A = \begin{bmatrix} l_{1,1} & 0 & 0 & 0 & 0 \\ l_{2,1} & l_{2,2} & 0 & 0 & 0 \\ \vdots & & l_{ii} & 0 & 0 \\ \vdots & & & \ddots & 0 \\ l_{n,1} & \dots & \dots & \dots & l_{n,n} \end{bmatrix} \begin{bmatrix} u_{1,1} & \dots & \dots & \dots & u_{1,n} \\ 0 & \ddots & & & \vdots \\ 0 & 0 & \ddots & & \vdots \\ 0 & 0 & 0 & \ddots & \vdots \\ 0 & 0 & 0 & 0 & u_{n,n} \end{bmatrix}$$

Exemple de factorisation de Gauss LU

Initialisation : $A^{(1)}=A$

Soit $A = \begin{bmatrix} 5 & 2 & 1 \\ 5 & -6 & 2 \\ -4 & 2 & 1 \end{bmatrix}$

Première étape : mettre des zéros dans la première colonne $A^{(1)}$

$$L_1^{-1} = \begin{bmatrix} 1 & & \\ -1 & 1 & \\ \frac{4}{5} & & 1 \end{bmatrix} \begin{array}{l} \text{ligne 1} \cdot (-1) + \text{ligne 2} \\ \text{ligne 1} \cdot \left(\frac{4}{5}\right) + \text{ligne 3} \end{array} \Rightarrow A^{(2)} = \begin{bmatrix} 5 & 2 & 1 \\ 0 & -8 & 1 \\ 0 & \frac{18}{5} & \frac{9}{5} \end{bmatrix}$$

Deuxième étape : mettre des zéros dans la deuxième colonne de $A^{(2)}$

$$L_2^{-1} = \begin{bmatrix} 1 & & \\ & 1 & \\ & \frac{9}{20} & 1 \end{bmatrix} \begin{array}{l} \\ \text{ligne 2} \cdot \left(\frac{9}{20}\right) + \text{ligne 3} \end{array} \Rightarrow U = A^{(3)} = \begin{bmatrix} 5 & 2 & 1 \\ & -8 & 1 \\ & 0 & \frac{9}{4} \end{bmatrix}$$

Factorisation

Domaine de validité : **toutes matrices**

- Algèbre : $L^{-1}A=U$
 - U matrice triangulaire supérieure,
 - L^{-1} matrice triangulaire inférieure, s'écrit comme le produit de

$$L^{-1} = L_{n-1}^{-1} L_{n-2}^{-1} \dots L_2^{-1} L_1^{-1} \Rightarrow L = L_1 L_2 \dots L_{n-2} L_{n-1}$$

Où L_k^{-1} est la matrice d'élimination élémentaire

Elle permet d'annuler les coefficients de la sous-colonne k :

$$A^{(k)}(k+1:n, k)$$

$$L_k^{-1} = \begin{bmatrix} 1 & & & & & \\ & \ddots & & & & \\ & & 1 & & & \\ & & & -l_{k+1,k} & & \\ & & & \vdots & & \\ \bigcirc & & & -l_{n,k} & & \bigcirc \end{bmatrix}$$

- Calcul de L_k est immédiate : $L_k = -L_k^{-1} + 2 Id$

Des problèmes de stabilité peut apparaître que si le pivot a_{kk} est trop petit en module.

Factorisation

Factorisation de Crout (LDL^T)

- Domaine de validité : *matrices symétriques définies positives*.

Symétrique → économie de place mémoire

- Algèbre : c' est la factorisation de Gauss qui maintient la structure symétrique

$$L^{-1} A L^{-T} = D$$

D matrice diagonale

L⁻¹ comme dans LU

- Intérêt : moins cher en espace, plus rapide en temps de calcul.

Algorithme de la factorisation LU

```
% A : matrice a factoriser de dim nxn
Id=eye(n); % Matrice Identite
L=Id;

for k = 1:n-1
% Construction de la matrice d'elimination
    inv_Lk=Id;
    for i = k+1:n
        inv_Lk(i,k)=-A(i,k)/A(k,k);
    end

% Construction de la matrice A(k+1) a partir de A(k)
    A=inv_Lk*A;

    Lk=-inv_Lk+2*Id; % Inverse de la matrice d'elimination
    L=L*Lk;
end
end

% matrice A(k+1) est la matrice triangulaire superieure
U=A;
```

Algorithme de la factorisation LU optimisé

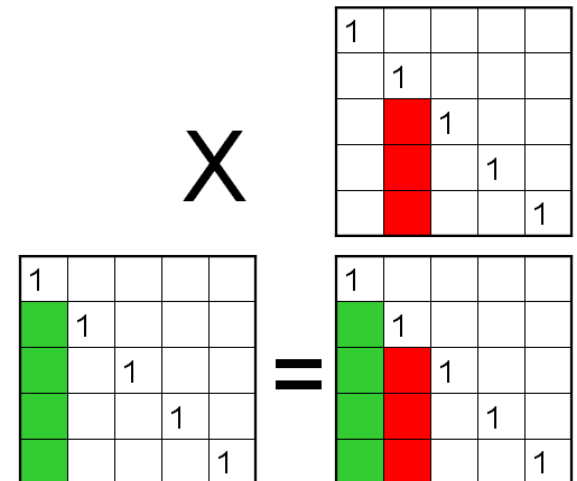
```
% A : matrice a factoriser de dim nxn
Id=eye(n); % Matrice Identite
L=Id;

for k = 1:n-1
% Construction de la matrice A(k+1) a partir de A(k)
% et de la matrice d elimination
    for i = k+1:n
        A(i,k+1:n) = A(i,k+1:n) - (A(i,k)/A(k,k))*A(k,k+1:n);
    end

% Stockage par colonne dans la matrice L
    L(k+1:n,k) = A(k+1:n,k)/A(k,k) ;

% les coefficients sous la diagonale
% de la colonne k sont nuls
    A(k+1:n,k)=0;
end

% matrice A(k+1) est la matrice triangulaire superieure
U=A;
```





Pour la matrice suivante : $A = \begin{bmatrix} 5 & 2 & 1 \\ 5 & -6 & 2 \\ -4 & 2 & 1 \end{bmatrix}$

On veut obtenir $x = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$

Calculer b

Programmer la méthode de factorisation de Gauss (falusp.m) puis la méthode de descente remontée (drlusp.m) sans pivot.

Comparer la solution numérique à la solution de référence

Stabilité

Le problème

Les erreurs d'arrondi font que la factorisation numérique diffère de la factorisation théorique. Au lieu de $A=LU$ on a $A=LU+E$

La factorisation est dite instable quand E devient comparable à L et/ou U . Dans ce cas, la solution numérique peut s'éloigner énormément de la solution exacte.

Le remède

- Cette instabilité est due aux divisions par des pivots “trop petits”.

Le remède consiste à changer de pivot.

- En pratique on utilise la **stratégie du pivot partiel** :

À la place du pivot $A^{(k)}(k,k)$ on prend $A^{(k)}(i_k,k)$ tel que

$$\left| A^{(k)}(i_k, k) \right| = \max \left\{ \left| A^{(k)}(i, k) \right| ; \quad i = k + 1, \dots, n \right\}$$

- Inconvénient : cette stratégie détruit la structure symétrique

Stabilité

Exemple $A = \begin{bmatrix} \varepsilon & 1 \\ 1 & 1 \end{bmatrix}$ ε : précision de votre calcul avec $b = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$

Solution analytique : $x_1 = 1/(1-\varepsilon) = 1$ $x_2 = (1-2\varepsilon)/(1-\varepsilon) = 1$

On applique **Gauss sans pivot** - matrice d'élimination $L^{-1} = \begin{bmatrix} 1 & 0 \\ -\frac{1}{\varepsilon} & 1 \end{bmatrix}$

$$\begin{bmatrix} \varepsilon & 1 \\ 0 & 1 - \frac{1}{\varepsilon} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 - \frac{1}{\varepsilon} \end{bmatrix} \quad \begin{bmatrix} 10^{-16} & 1 \\ 0 & -10^{16} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ -10^{16} \end{bmatrix}$$

si la précision de la machine n'est pas suffisante $(1-1/\varepsilon) = -1/\varepsilon$, on obtient $x_2 = 1$ et $x_1 = 0$

On applique **Gauss avec pivot** - $A = \begin{bmatrix} 1 & 1 \\ \varepsilon & 1 \end{bmatrix}$ matrice d'élimination : $L^{-1} = \begin{bmatrix} 1 & 0 \\ -\varepsilon & 1 \end{bmatrix}$

$$\begin{bmatrix} 1 & 1 \\ 0 & 1 - \varepsilon \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 - 2\varepsilon \end{bmatrix} \quad \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

si la précision de la machine n'est pas suffisante $(1-\varepsilon) = 1$, mais on obtient $x_1 = 1$ et $x_2 = 1$

Démonstration matlab : stablu(epsilon)

Conditionnement

Le problème

Soit δb une perturbation de b (par exemple, une erreur de mesure), et δx la perturbation résultante de la solution x de $Ax=b$ et $A(x + \delta x) = b + \delta b$

La question est alors :

est-ce que $\frac{\|\delta b\|}{\|b\|}$ est du même ordre de grandeur que $\frac{\|\delta x\|}{\|x\|}$?

Pour une matrice bien conditionnée, la réponse est OUI.

Le remède

PAS DE REMÈDE !

Mesure du conditionnement $\frac{\|\delta x\|}{\|x\|} \leq \text{cond}(A) \frac{\|\delta b\|}{\|b\|}$

Le conditionnement $\text{cond}(A)$ dépend de la norme. Dans le cas de la norme euclidienne il n'est rien d'autre que le rapport des valeurs singulières extrêmes (la plus grande sur la plus petite).

$$\text{cond}([A]) = \frac{\lambda_{\text{Max}}([A])}{\lambda_{\text{Min}}([A])}$$

Conditionnement

Exemple

$$\begin{bmatrix} 10 & 7 & 8 & 7 \\ 7 & 5 & 6 & 5 \\ 8 & 6 & 10 & 9 \\ 7 & 5 & 9 & 10 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \end{bmatrix} = \begin{bmatrix} 32 \\ 23 \\ 33 \\ 31 \end{bmatrix} \text{ dont la solution est } \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

Pour un second membre légèrement modifié on obtient la solution suivante :

$$b = \begin{bmatrix} 32.5 \\ 22.9 \\ 33.1 \\ 30.9 \end{bmatrix} \Rightarrow u = \begin{bmatrix} 19.2 \\ -29.0 \\ 8.5 \\ -3.5 \end{bmatrix}$$

Erreur relative de 1/200 sur le 2nd membre $\rightarrow$ erreur relative de 10/1 sur le résultat
Soit une amplification de l'ordre de 2000 !

Exemple troublant car erreurs sur les données d'un ordre de grandeur satisfaisant en sciences expérimentales

Exercice : conditionnement



Soit
$$\begin{bmatrix} 10 & 7 & 8 & 7 \\ 7 & 5 & 6 & 5 \\ 8 & 6 & 10 & 9 \\ 7 & 5 & 9 & 10 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \end{bmatrix} = \begin{bmatrix} 32 \\ 23 \\ 33 \\ 31 \end{bmatrix} \text{ dont la solution est } \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

Appliquer une perturbation au 2nd membre : $b = \begin{bmatrix} 0.5 \\ -0.1 \\ 0.1 \\ -0.1 \end{bmatrix}$

Calculer l'erreur relative sur b, puis sur la solution.

Faire le rapport des erreurs relatives.

Calculer les valeurs propres de la matrice A.

Calculer le conditionnement de A.

Mise en œuvre

Mode de stockage des matrices

Des variantes des algorithmes précédents existent pour traiter des matrices creuses (contenant beaucoup de zéros) ou bandes.

Seuls les coefficients non nuls sont stockés.

Ainsi, on aura :

- une variante de LDL^T pour les matrices tridiagonales symétriques (cas particulier de matrice bande) utilisant trois vecteurs pour stocker la matrice,
- une variante de LU pour les matrices bandes utilisant autant de vecteurs pour stocker la matrice que le nombre de bandes (plus un espace additionnel pour pivoter),
- une variante de LU pour les matrices creuses

La contre-partie du gain en stockage est un temps d'accès aux éléments de matrice plus élevé que l'accès direct $A(i,j)$.

L'accès aux éléments dans une matrice creuse nécessite un double adressage.

Guide

Les matrices réelles symétriques définies positives ne posent pas de problèmes. Utiliser la méthode LDL^T sans pivot

Sous Matlab : `[L, D]=ldl(A); A = L*D*L'` ;

Dans les autres cas, utiliser LU avec pivot. Ne pas prendre le risque d'utiliser un LU sans pivot.

Sous Matlab : `[L, U, P]=lu(A); % P matrice de permutation`
`A = P'*L*U;`

Choisir toujours la variante de mise en œuvre adaptée aux propriétés structurelles (stockage spécialisé pour matrices bandes). De la sorte, on peut “faire passer” des problèmes plus gros et ce plus vite

Vérifier le conditionnement